香港中文大學
The Chinese University of Hong Kong

ROSE5710 Fundamental Robotics and Systems

Lecture Set 0: Course Introduction

Darwin Lau

2026-27

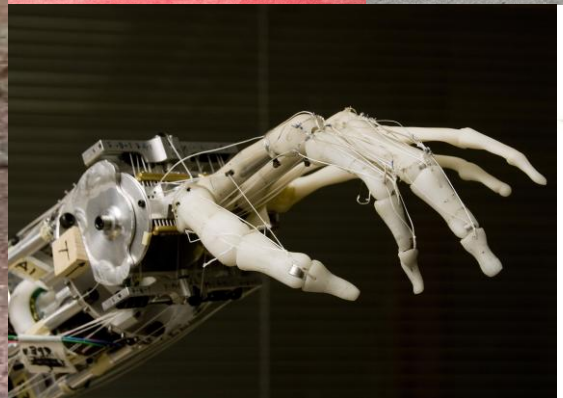
Robotics in Real-Life













香港中文大學
The Chinese University of Hong Kong

Robotics in MAEG5755

Because joints 2 and 3 are always parallel, multiplying ${}^1_2T$ and ${}^2_3T$ first and then applying sum-of-angle formulas will yield a somewhat simpler final expression. This can be done whenever two rotational joints have parallel axes and we have

$$ {}^1_3T = {}^1_2T\,{}^2_3T = \begin{bmatrix} c_{23} & -s_{23} & 0 & a_2c_2 \\ 0 & 0 & 1 & d_3 \\ -s_{23} & -c_{23} & 0 & -a_2s_2 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{3.12} $$

where we have used the sum-of-angle formulas (from Appendix A):

$$ c_{23} = c_2c_3 - s_2s_3, $$
$$ s_{23} = c_2s_3 + s_2c_3. $$

Then we have

$$ {}^1_6T = {}^1_3T\,{}^3_6T = \begin{bmatrix} {}^1r_{11} & {}^1r_{12} & {}^1r_{13} & {}^1p_x \\ {}^1r_{21} & {}^1r_{22} & {}^1r_{23} & {}^1p_y \\ {}^1r_{31} & {}^1r_{32} & {}^1r_{33} & {}^1p_z \\ 0 & 0 & 0 & 1 \end{bmatrix}, $$

where

$$ \begin{aligned} {}^1r_{11} &= c_{23}[c_4c_5c_6 - s_4s_6] - s_{23}s_5s_6, \\ {}^1r_{21} &= -s_4c_5c_6 - c_4s_6, \\ {}^1r_{31} &= -s_{23}[c_4c_5c_6 - s_4s_6] - c_{23}s_5c_6, \\ {}^1r_{12} &= -c_{23}[c_4c_5s_6 + s_4c_6] + s_{23}s_5s_6, \\ {}^1r_{22} &= s_4c_5s_6 - c_4c_6, \\ {}^1r_{32} &= s_{23}[c_4c_5s_6 + s_4c_6] + c_{23}s_5s_6, \\ {}^1r_{13} &= -c_{23}c_4s_5 - s_{23}c_5, \\ {}^1r_{23} &= s_4s_5, \\ {}^1r_{33} &= s_{23}c_4s_5 - c_{23}c_5, \\ {}^1p_x &= a_2c_2 + a_3c_{23} - d_4s_{23}, \\ {}^1p_y &= d_3, \\ {}^1p_x &= -a_3s_{23} - a_2s_2 - d_4c_{23}. \end{aligned} \tag{3.13} $$

Finally, we obtain the product of all six link transforms:

$$ {}^0_6T = {}^0_1T\,{}^1_6T = \begin{bmatrix} r_{11} & r_{12} & r_{13} & p_x \\ r_{21} & r_{22} & r_{23} & p_y \\ r_{31} & r_{32} & r_{33} & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix}. $$

Here,

$$ \begin{aligned} r_{11} &= c_1[c_{23}(c_4c_5c_6 - s_4s_5) - s_{23}s_5c_5] + s_1(s_4c_5c_6 + c_4s_6), \\ r_{21} &= s_1[c_{23}(c_4c_5c_6 - s_4s_6) - s_{23}s_5c_6 - c_1(s_4c_5c_6 + c_4s_6), \\ r_{31} &= -s_{23}(c_4c_5c_6 - s_4s_6) - c_{23}s_5c_6, \\ r_{12} &= c_1[c_{23}(-c_4c_5s_6 - s_4c_6) + s_{23}s_5s_6] + s_1(c_4c_6 - s_4c_5s_6), \\ r_{22} &= s_1[c_{23}(-c_4c_5s_6 - s_4c_6) + s_{23}s_5s_6] - c_1(c_4c_6 - s_4c_5s_6), \\ r_{32} &= -s_{23}(-c_4c_5s_6 - s_4c_6) + c_{23}s_5s_6, \\ r_{13} &= -c_1(c_{23}c_4s_5 + s_{23}c_5) - s_1s_4s_5, \\ r_{23} &= -s_1(c_{23}c_4s_5 + s_{23}c_5) + c_1s_4s_5, \\ r_{33} &= s_{23}c_4s_5 - c_{23}c_5, \\ p_x &= c_1[a_2c_2 + a_3c_{23} - d_4s_{23}] - d_3s_1, \\ p_y &= s_1[a_2c_2 + a_3c_{23} - d_4s_{23}] + d_3c_1, \\ p_z &= -a_3s_{23} - a_2s_2 - d_4c_{23}. \end{aligned} \tag{3.14} $$

Equations (3.14) constitute the kinematics of the PUMA 560. They specify how to compute the position and orientation of frame {6} relative to frame {0} of the robot. These are the basic equations for all kinematic analysis of this manipulator.

The Yasukawa Motoman L-3

The Yasukawa Motoman L-3 is a popular industrial manipulator with five degrees of freedom (Fig. 3.22). Unlike the examples we have seen thus far, the Motoman is not a simple open kinematic chain, but rather makes use of two linear actuators coupled to links 2 and 3 with four-bar linkages. Also, through a chain drive, joints 4 and 5 are operated by two actuators in a differential arrangement.

In this example, we will solve the kinematics in two stages. First, we will solve for joint angles from actuator positions; second, we will solve for Cartesian position and orientation of the last link from joint angles. In this second stage, we can treat the system as if it were a simple open-kinematic-chain $5R$ device.

Figure 3.23 shows the linkage mechanism that connects actuator number 2 to links 2 and 3 of the robot. The actuator is a linear one that directly controls the length of the segment labeled DC. Triangle ABC is fixed, as is the length BD. Joint 2 pivots about point B, and the actuator pivots slightly about point C as the linkage moves. We give the following names to the constants (lengths and angles) associated with actuator 2:

$$ \gamma_2 = AB, \phi_2 = AC, \alpha_2 = BC, $$
$$ \beta_2 = BD, \Omega_2 = \angle JBD, l_2 = BJ, $$

Now we'll begin with coding. This is pretty straightforward Python code – we won't get fancy until later:

1. We start by importing our libraries. It is not surprising that we start with the `time` module. We also will use the `mean` function from `numpy` (Python numerical analysis) and `matplotlib` to draw our graph at the end. We will also be doing some math calculations to simulate our processing and create a load on the frame rate:

```
import time
from numpy import mean
import matplotlib.pyplot as plt
import math
#
```

2. Now we have some parameters to control our test. This is where you can experiment with different timings. Our basic control is `FRAMERATE` – how many updates per second do we want to try? Let's start with `30`, as we did in the example we discussed earlier:

```
# set our frame rate - how many cycles per second to run our
loop?
FRAMERATE = 30
# how long does each frame take in seconds?
FRAME = 1.0/FRAMERATE
# initialize myTimer
# This is one of our timer variables where we will store the
clock time from the operating system.
myTimer = 0.0
```

3. The duration of the test is set by the `counter` variable. The time the test will take is the `FRAME` time times the number of cycles in `counter`. In our example, 2,000 frames divided by 30 fps is 66.6 seconds, or a bit over a minute to run the test:

```
# how many cycles to test? counter*FRAME = runtime in seconds
counter = 2000
```

We will be controlling our timing loop in two ways:

- We will first measure the amount of time it takes to perform the calculations for this frame. We have a stub of a program with some trigonometry functions we will call to put a load on the computer. Robot control functions, such as computing the angles needed in a robot arm, need lots of trig to work. This is available from `import math` in the header of the program.

香港中文大學
The Chinese University of Hong Kong

Robotics in MAEG5755

Here,

$$
\begin{aligned}
r_{11} &= c_1[c_{23}(c_4c_5c_6 - s_4s_6) - s_{23}s_5c_6] + s_1(s_4c_5c_6 + c_4s_6),\\
r_{21} &= s_1[c_{23}(c_4c_5c_6 - s_4s_6) - s_{23}s_5c_6 - c_1(s_4c_5c_6 + c_4s_6),\\
r_{31} &= -s_{23}(c_4c_5c_6 - s_4s_6) - c_{23}s_5c_6,\\
\\
r_{12} &= c_1[c_{23}(-c_4c_5s_6 - s_4c_6) + s_{23}s_5s_6] + s_1(c_4c_6 - s_4c_5s_6),\\
r_{22} &= s_1[c_{23}(-c_4c_5s_6 - s_4c_6) + s_{23}s_5s_6] - c_1(c_4c_6 - s_4c_5s_6),\\
r_{32} &= -s_{23}(-c_4c_5s_6 - s_4c_6) + c_{23}s_5s_6,\\
\\
r_{13} &= -c_1(c_{23}c_4s_5 + s_{23}c_5) - s_1s_4s_5,\\
r_{23} &= -s_1(c_{23}c_4s_5 + s_{23}c_5) + c_1s_4s_5,\\
r_{33} &= s_{23}c_4s_5 - c_{23}c_5,\\
\\
p_x &= c_1[a_2c_2 + a_3c_{23} - d_4s_{23}] - d_3s_1,\\
p_y &= s_1[a_2c_2 + a_3c_{23} - d_4s_{23}] + d_3c_1,\\
p_z &= -a_3s_{23} - a_2s_2 - d_4c_{23}. \qquad (3.14)
\end{aligned}
$$

Equations (3.14) constitute the kinematics of the PUMA 560. They specify how to compute the position and orientation of frame {6} relative to frame {0} of the robot. These are the basic equations for all kinematic analysis of this manipulator.

The Yasukawa Motoman L-3

The Yasukawa Motoman L-3 is a popular industrial manipulator with five degrees of freedom (Fig. 3.22). Unlike the examples we have seen thus far, the Motoman is not a simple open kinematic chain, but rather makes use of two linear actuators coupled to links 2 and 3 with four-bar linkages. Also, through a chain drive, joints 4 and 5 are operated by two actuators in a differential arrangement.

In this example, we will solve the kinematics in two stages. First, we will solve for joint angles from actuator positions; second, we will solve for Cartesian position and orientation of the last link from joint angles. In this second stage, we can treat the system as if it were a simple open-kinematic-chain 5*R* device.

Figure 3.23 shows the linkage mechanism that connects actuator number 2 to links 2 and 3 of the robot. The actuator is a linear one that directly controls the length of the segment labeled *DC*. Triangle *ABC* is fixed, as is the length *BD*. Joint 2 pivots about point *B*, and the actuator pivots slightly about point *C* as the linkage moves. We give the following names to the constants (lengths and angles) associated with actuator 2:

$$\gamma_2 = AB, \phi_2 = AC, a_2 = BC,$$

$$\beta_2 = BD, \Omega_2 = \angle JBD, l_2 = BJ,$$

2.2 · Working in Two Dimensions (2D)

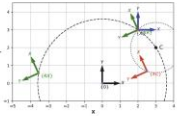


Fig. 2.13 The frame {X} is rotated by 2 radians about {0} to give frame {RX}, about {X} to give {XR}, and about point C to give frame {XC}

To understand how this works we can unpack what happens when we apply TC to T3. Since $\mathbf{T}_C$ premultiplies $\mathbf{T}_1$, the first transform that is applied to {X} is the right-most one, `trotz(1.2)`, which performs an origin shift that places C at the origin of the reference frame. Then we apply $\mathbf{T}_R$, a pure rotation which rotates the shifted version of {X} about the origin, which is where C now is – therefore {X} is rotated about C. Finally, we apply `transl(3)` which is the inverse origin shift, that places C back at its original position and {X} to its final pose. This transformation is an example of a conjugation but its construction was somewhat elaborate. Premultiplication of matrices is discussed in ▶ Sect. 2.4.1. A more intuitive way to achieve this same result is by using twists which we introduce in ▶ Sect. 2.2.2.4.

2.2.2.3 Matrix Exponential for Pose

We can take the logarithm of the SE(2) matrix TC from the previous example

```
>>> L = linalg.logm(TC)
array([[ 0.    , -1.2   ,  4.    ],
       [ 1.2   ,  0.    , -4.    ],
       [ 0.    ,  0.    ,  0.    ]])
```

and the result is an augmented skew-symmetric matrix: the upper-left corner is a 2×2 skew-symmetric matrix: the upper right column is a 2-vector, and the bottom row is zero. The three unique elements can be unpacked

```
>>> S = vex(L)
array([ 4.  , -4.  ,  2.  ])
```

and the last element is the rotation angle.

Exponentiating ξ yields the original SE(2) matrix, and ξ can be reconstructed from just the three elements of S

```
>>> linalg.expm(skew(S))
array([[ -0.4161,  -0.9093,   6.063 ],
       [  0.9093,  -0.4161,   0.1044],
       [  0.    ,   0.    ,   1.    ]])
```

In general, we can write

$$\mathbf{T} = e^{[S]} \in \mathbf{SE}(2)$$

where $S \in \mathbb{R}^3$ and $[\cdot]: \mathbb{R}^3 \mapsto \mathfrak{se}(2) \subset \mathbb{R}^{3\times3}$. The logarithm and exponential can be computed efficiently using the Toolbox functions `trlog2` and `trexp2` respectively. The vector S is a twist vector which we will discuss next.










```
# set our frame rate - how many cycles per second to run our
loop?
FRAMERATE = 30
# how long does each frame take in seconds?
FRAME = 1.0/FRAMERATE
# initialize myTimer
# This is one of our timer variables where we will store the
clock time from the operating system.
myTimer = 0.0
```

3. The duration of the test is set by the `counter` variable. The time the test will take is the `FRAME` time times the number of cycles in `counter`. In our example, 2,000 frames divided by 30 fps is 66.6 seconds, or a bit over a minute to run the test:

```
# how many cycles to test? counter*FRAME = runtime in seconds
counter = 2000
```

Course Syllabus

Lecture Set 1	Introduction to Robotics Development
Lecture Set 2	Robot Hardware and Software
Lecture Set 3	Robot Modelling Foundations
Lecture Set 4	Position-level Forward Kinematics
Lecture Set 5	Position-level Inverse Kinematics
Lecture Set 6	Trajectory Generation
Lecture Set 7	Velocity-level Kinematics
Lecture Set 8	Dynamics and Control
Lecture Set 9	Robot Learning

Textbooks and References

Textbooks

1. Robotics, Vision and Control, 3rd edition, Peter Corke, Springer, 2023
2. Intelligence for Robotics, 2nd edition, Francis X. Govers III, Packt Publishing, 2024

Recommended readings

3. Introduction to Autonomous Robots: Mechanisms, Sensors, Actuators, and Algorithms, Nikolaus Correll, Bradley Hayes, Christoffer Heckman, and Alessandro Roncone, MIT Press, 2022
4. Introduction to AI Robotics, 2nd edition, Robin R. Murphy, MIT Press, 2019

Class Information and Assessment

Lectures

- Wednesday 6:30 – 9:15pm, LSK LT2
- Consultation: after class, e-mail, make in-person/zoom appointment

Assessment

- Mid-term test: 30% (November TBD)
- Assignments 1 and 2: 40% (20% each)
- Project: 30%

Course Assignment and Project

Project

- Practice the “theory” learning in the course
- Program the robot arm using ROS2
- Implement trajectory generation and kinematics
- Implement robot control
- Implement robot learning

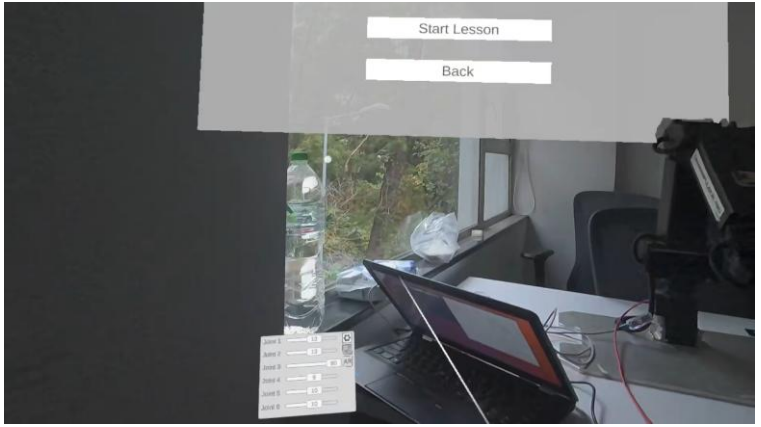
Assignments

- Exercises for the course
- Progress for the project

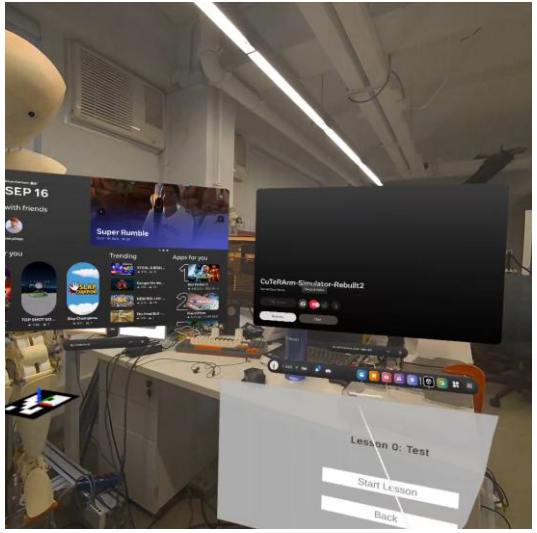
Other Materials

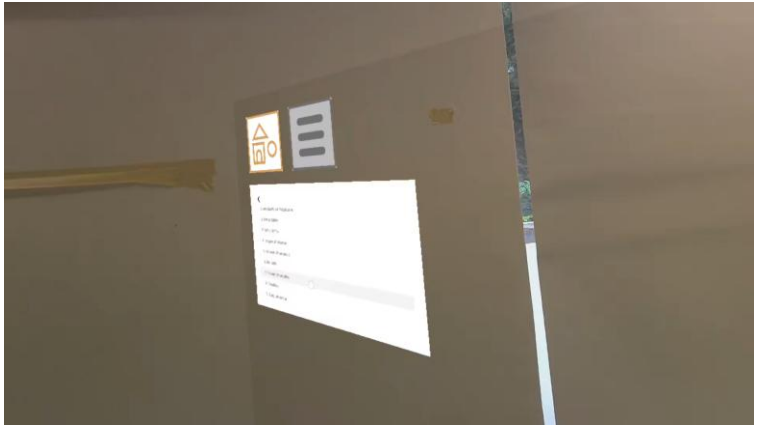
Robot Virtual Online

- http://maeg3060.mae.cuhk.edu.hk/virtualarm



Self-Learning Modules





Audio is from here =)